

Antes do git: versionar na mão

PROJETO

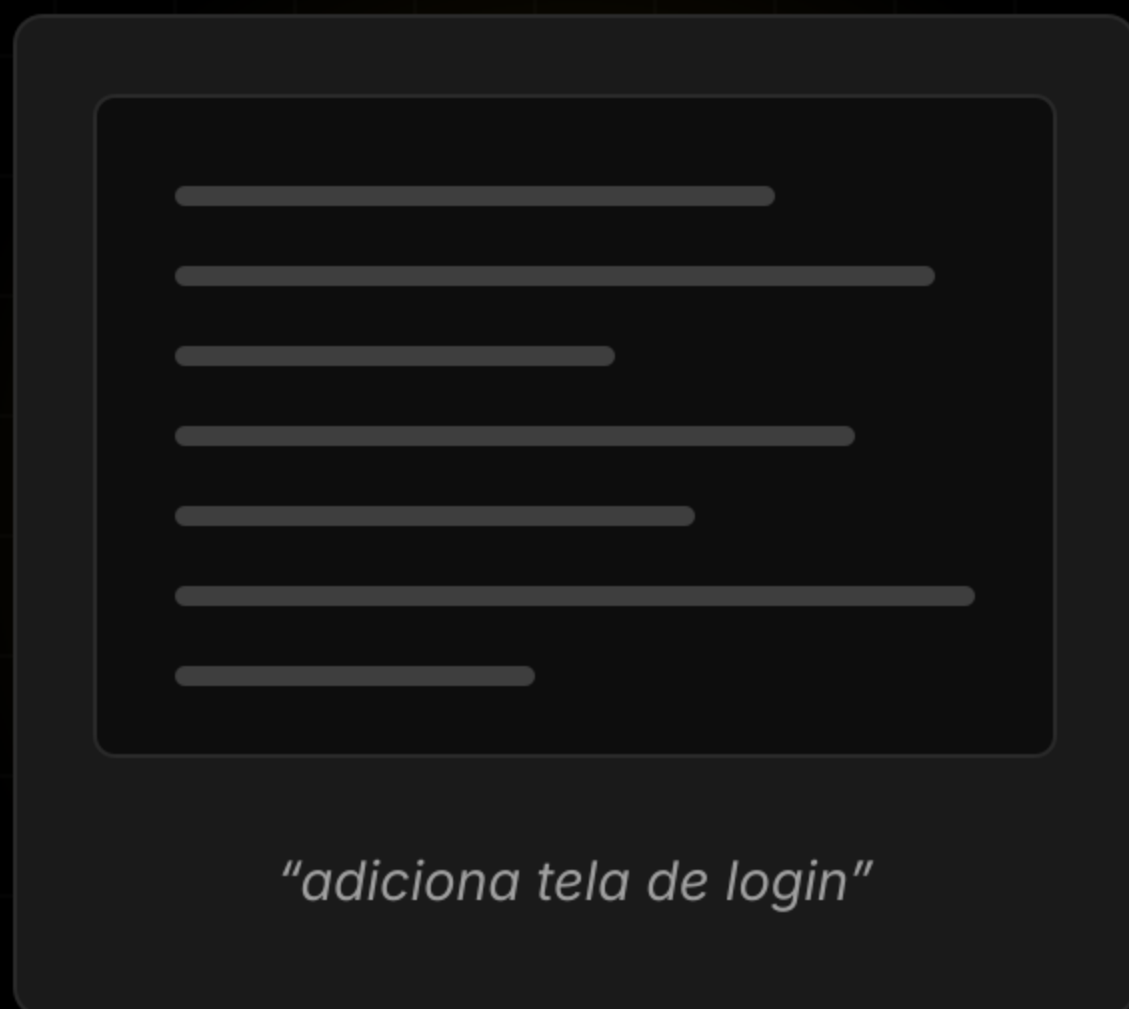
- ☐ app.js
- ☐ app_v2.js
- ☐ app_final.js
- ☒ **app_final_agora_vai.js**
- ☐ app_velho_nao_apagar.js

QUATRO PERGUNTAS SEM RESPOSTA

- ✗ o que mudou entre as versões?
- ✗ quem mudou?
- ✗ quando quebrou?
- ✗ por que mudou?

Funciona para um arquivo. Para um projeto inteiro, vira caos.

Commit: a foto do código



TODA FOTO REGISTRA

MENSAGEM

o que mudou, escrito por você

AUTOR

quem tirou a foto

DATA

quando a foto foi tirada

IDENTIFICADOR

o número de série, único

IMUTÁVEL

depois de tirada, a foto não muda

Ficou ruim? Tira uma foto nova. A antiga continua no álbum.

Repo: o álbum onde as fotos moram



A mesma linha do tempo, em dois lugares: sua máquina e um servidor.

A plataforma muda de nome; o conceito é um só.

Só com isso, a rede de segurança básica

Quebrou algo?

volta para a foto anterior

O que mudou?

compara duas fotos

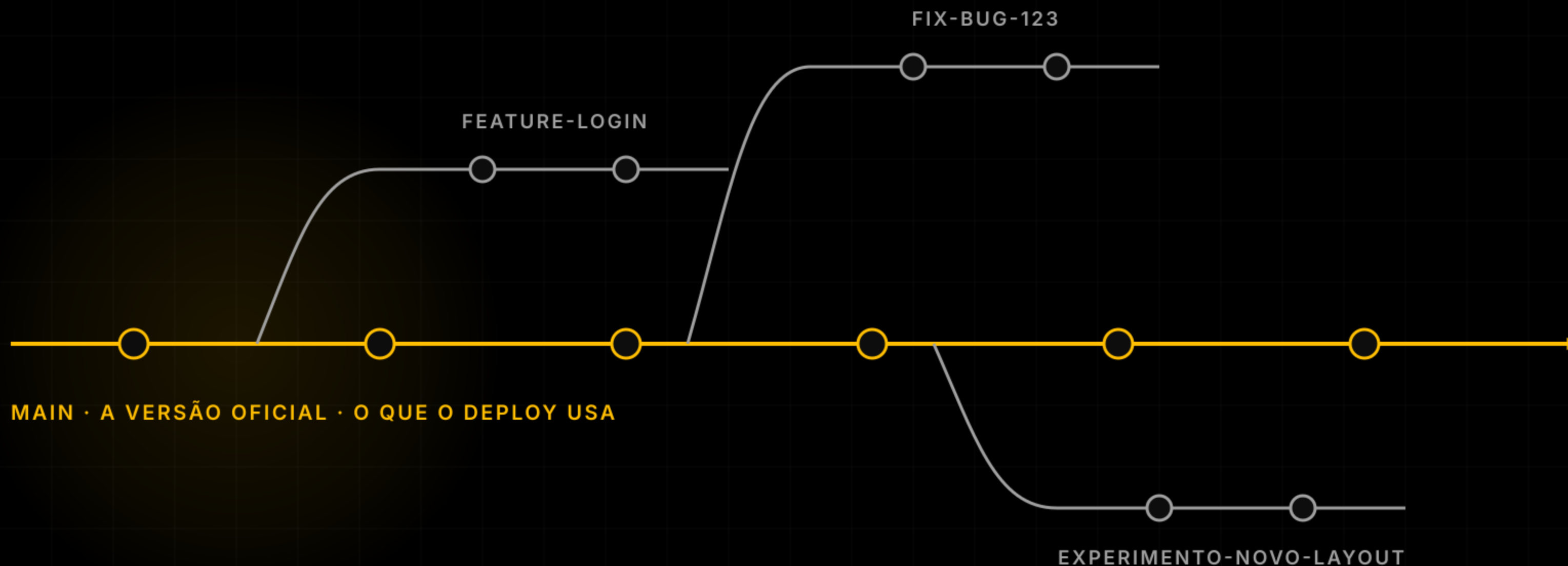
Quem fez isso?

olha o autor e a data

Isso já resolve 80% do problema.

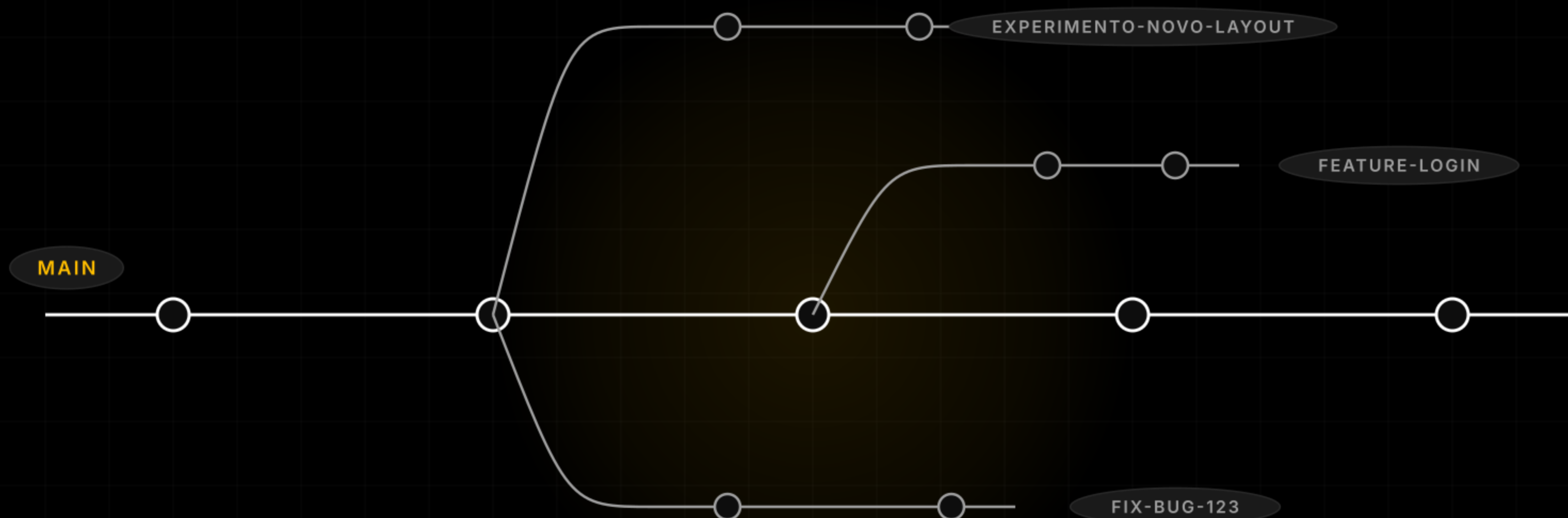
Falta uma coisa: testar algo arriscado sem estragar a linha principal.

Branch: um ramal para cada trabalho



A main fica intocada enquanto cada ramal experimenta.

Uma main, vários ramos



Criar branch não copia o projeto: é um ponteiro para uma foto que já existe.

Branch não copia nada. É um ponteiro com nome.

Merge: a confluência dos rios



Depois do merge, a main tem tudo que estava na branch.

Dois rios se encontram e viram um só.

Conflito: o git não decide sozinho

UM ARQUIVO · DUAS MUDANÇAS NO MESMO LUGAR

função de login:

<<<<<< **sua versão**

botão azul, texto "Entrar"

=====

botão verde, texto "Acessar"

>>>>>> **versão do colega**

o git parou aqui e marcou o arquivo

RESOLVER É MANUAL

- ✓ abrir, ler as duas versões, decidir e apagar as marcas
- ✗ aceitar "resolvido" da IA sem olhar o arquivo

Conflito mal resolvido é bug silencioso.

Conflito bem resolvido é coisa de humano.

Pull request: a sala de revisão

Adicionar tela de login

PULL REQUEST

DIFF · LINHA POR LINHA

- botão sem verificação de senha
- + verifica a senha antes de entrar
- + mensagem de erro clara

3 ARQUIVOS MODIFICADOS · 2 COMMITS

COMENTÁRIO DO REVISOR

"por que você fez assim?"

RESPOSTA

"a antiga deixava passar senha vazia."

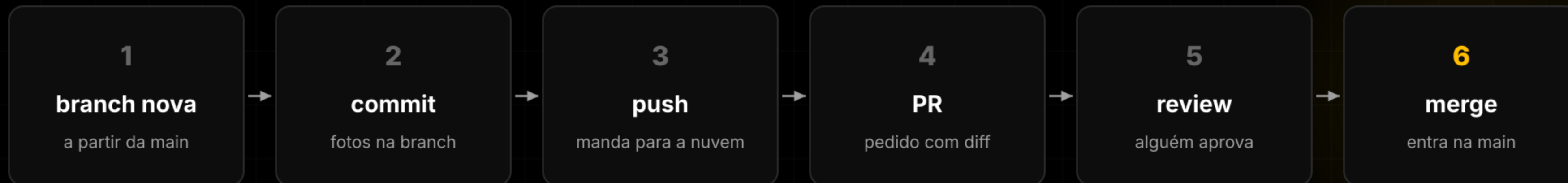
"Tenho uma branch pronta. Olha o que mudei. Posso juntar na main?"

Review: o portão antes da main



Entre "eu fiz" e "está na main", sempre um segundo par de olhos.

O fluxo completo, sempre nessa ordem



Dez pessoas, dez branches. Todas convergem na main pelo mesmo portão.

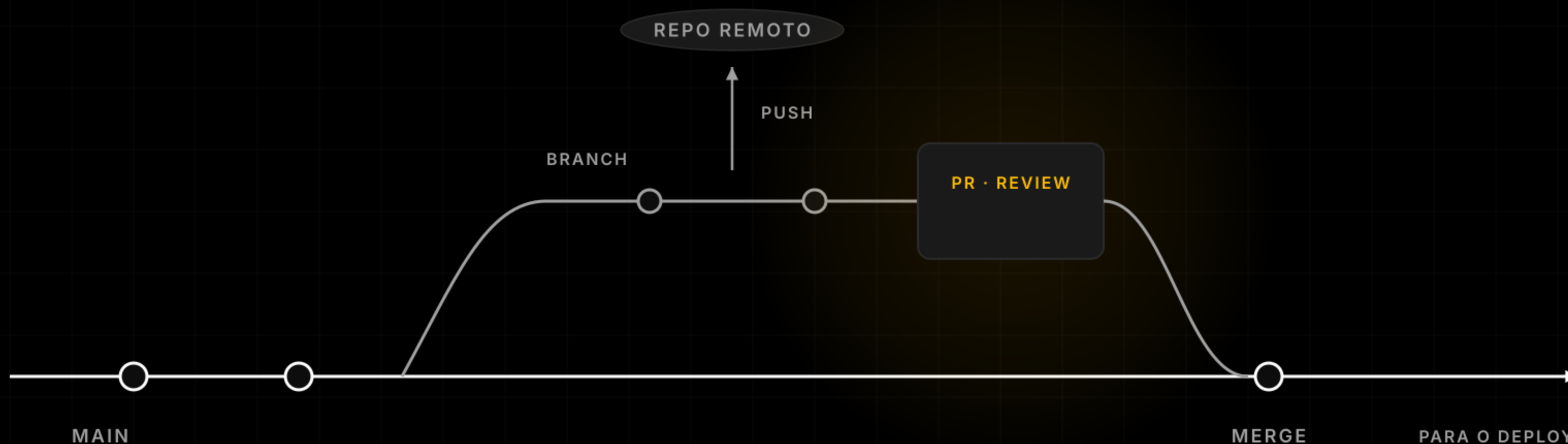
Esse é o coração do trabalho profissional com código.

O deploy usa o repo, não seu computador



Sem push, seu código não existe para o resto do mundo.

A rede de segurança inteira



commit é a foto · branch é o ramal · PR e review são o portão · merge é a confluência

Código novo chega na main de forma controlada, revisada, sem surpresa.